



Universidade Federal de Ouro Preto
Sistemas Operacionais

BabyBuddy

Gustavo Guimarães de Oliveira Dias

João Monlevade, 17 de Julho de 2026



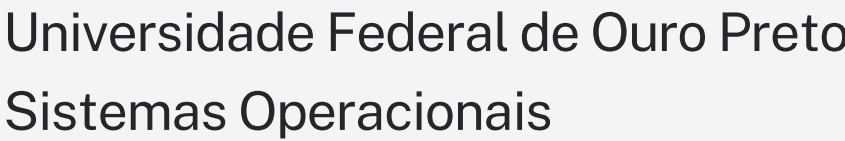
Universidade Federal de Ouro Preto
Sistemas Operacionais

Sobre o framework

Conceitos Fundamentais

- Framework desenvolvido para acompanhamento de rotinas do bebê
- Desenvolvido em Python e Django, estilizado com HTML, CSS e JavaScript
- Possui versão mobile





Sobre o framework





Sobre o framework

Baby Buddy

Dashboard

Timeline

Children

Measurements

Activities

Timers

Home / Feedings

Feedings

Add Feeding

Add Bottle Feeding

Filters

Actions	Date	Method	Type	Amt.	Duration
	Today, 6:16 p.m.	Bottle	Formula		
	Today, 3:37 a.m.	Bottle	Formula		1 hour, 0 minutes
	Today, 3:37 a.m.	Bottle	Breast milk	100.0	

Baby Buddy

Dashboard

Timeline

Children

Measurements

Activities

Timers

Home / Feedings / Add

Add a Feeding

Child

Gustavo Dias

Start time

17/07/2026 22:18:06

End time

17/07/2026 22:18:06

Type

Breast milk

Formula

Fortified breast milk

Solid food

Method

Bottle

Left breast

Right breast

Both breasts

Parent fed

Self fed

Amount

Advanced Fields

Submit



Universidade Federal de Ouro Preto
Sistemas Operacionais

Arquitetura





Universidade Federal de Ouro Preto
Sistemas Operacionais

Arquitetura

Conceitos Fundamentais

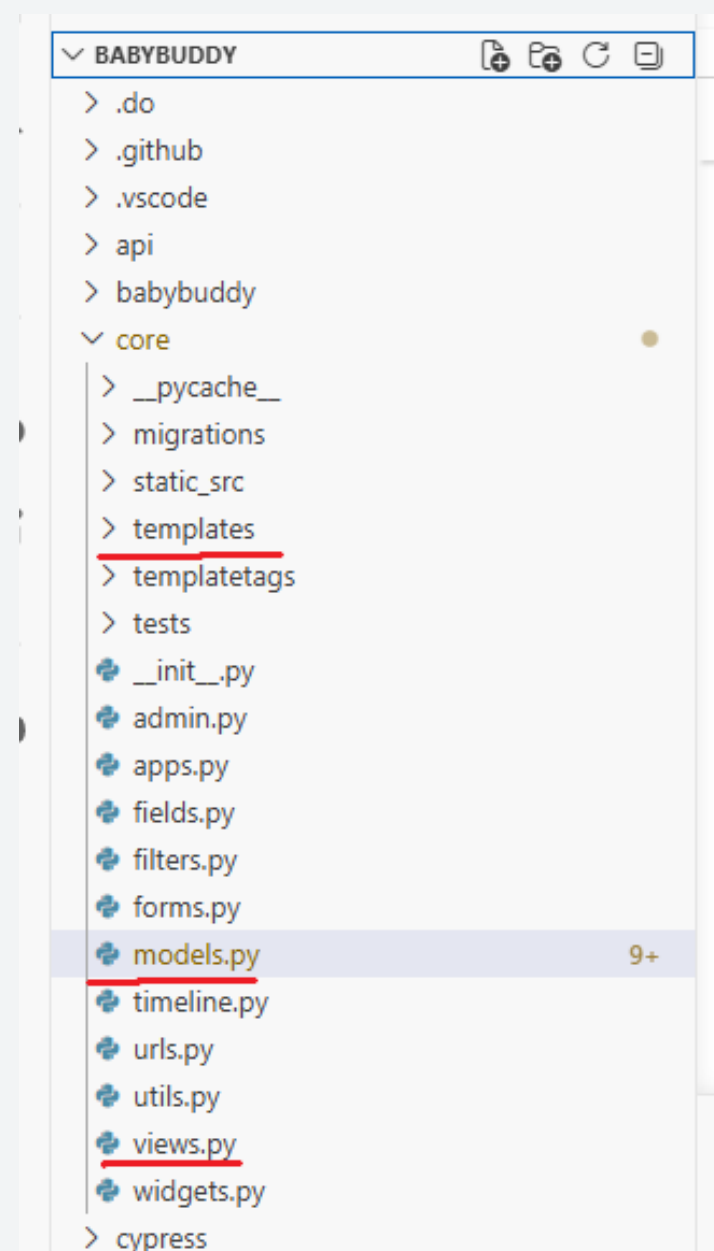
- Arquitetura MVT Model - View - Template
- Arquitetura própria do Django
- Similar a arquitetura MVC Model - View - Controller
- Separada pela parte lógica (model), parte responsável por “formatar” os dados a serem exibidos (view) e o que será mostrado ao usuário (template)





Universidade Federal de Ouro Preto
Sistemas Operacionais

Arquitetura





UFOP

Universidade Federal de Ouro Preto
Sistemas Operacionais

Issue Trabalhada





Universidade Federal de Ouro Preto
Sistemas Operacionais

Issue Trabalhada

Interpretação do Bug

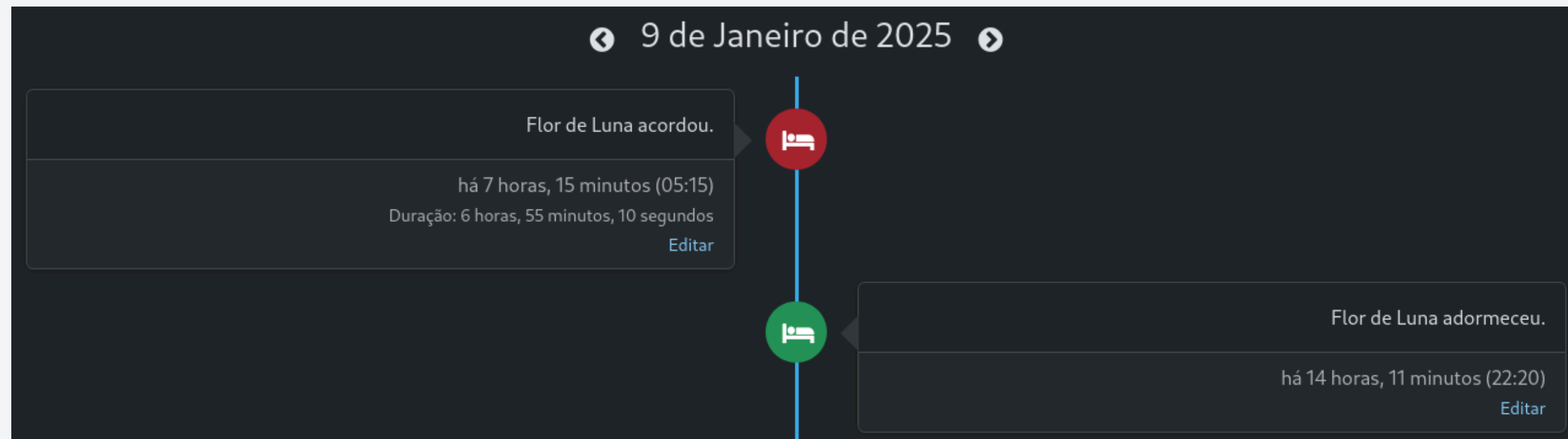
- Issue #942 do projeto
- Foi reportado um bug por um usuário em que se você cadastrasse um tempo de sono entre um dia e outro, ambos os registros seriam incluídos no dia de início na Timeline
- Ex: início do sono às 22:20 de 09/01 e término às 05:15 de 10/01, ambos seriam incluídos na data 09/01





Universidade Federal de Ouro Preto
Sistemas Operacionais

Issue Trabalhada





Universidade Federal de Ouro Preto
Sistemas Operacionais

UFOP

Issue Trabalhada

Solução do Bug

- Importado o objeto Q do Django, que permite fazer consultas no banco de dados combinadas com AND, OR e NOT.
- Incrementado a lógica por trás das consultas dos registros (anteriormente só conferia se existia ou não os registros).
- Criado o helper `_is_in_range`, para verificar se o registro pertence a timeline selecionada.
- Criado a função `_duration_instances_for_day`, que verifica e ordena os registros distintos cadastrados por horário e por criança na timeline.





Issue Trabalhada

Funções citadas

```
def _is_in_range(value, min_date, max_date):  
    return min_date <= value <= max_date  
  
def _duration_instances_for_day(model, min_date, max_date, child=None):  
    instances = model.objects.filter(  
        Q(start__range=(min_date, max_date)) | Q(end__range=(min_date, max_date))  
    ).distinct()  
  
    if child:  
        instances = instances.filter(child=child)  
  
    return instances.order_by("-start")
```





Universidade Federal de Ouro Preto
Sistemas Operacionais

UFOP

Refatoração Sugerida





Code Smells

Interpretação

- O primeiro Code Smell que identifiquei foi em decorrência da análise da própria Issue.
- A partir da interpretação e solução desse bug, as classes TummyTime, Sleep e Feeding também foram alteradas, já que possuíam uma implementação bem semelhante.
- Todas elas passaram a ter `_is_in_range` e `_duration_instances_for_day`, para evitar que o mesmo bug ocorresse quando o usuário cadastrasse um evento em qualquer uma dessas classes.
- Code Smell do tipo DRY - Don't Repeat Yourself.





Universidade Federal de Ouro Preto
Sistemas Operacionais

UFOP

Code Smells

Interpretação

- O segundo Code Smell observado foi em relação a função `_add_feedings`.
- A função possuía muitas responsabilidades; consulta em banco de dados, calculo de tempo entre um evento e outro, formatação de detalhes, criação de eventos, dentre outros.
- Para resolvê-lá, a função foi parcionada em outras menores, a fim de diminuir a sua responsabilidade.
- Code Smell do tipo SRP - Single Responsibility Principle.





UFOP

Code Smells

```
129 def _add_feedings(min_date, max_date, events, child=None):
130     # Ensure first feeding has a previous.
131     yesterday = min_date - timedelta(days=1)
132     prev_start = None
133
134     instances = Feeding.objects.filter(start__range=(yesterday, max_date)).order_by(
135         "start"
136     )
137     if child:
138         instances = instances.filter(child=child)
139     for instance in instances:
140         details = []
141         if instance.notes:
142             details.append(instance.notes)
143         time_since_prev = None
144         if prev_start:
145             time_since_prev = timesince.timesince(prev_start, now=instance.start)
146         prev_start = instance.start
147         if instance.start < min_date:
148             continue
149         edit_link = reverse("core:feeding-update", args=[instance.id])
150         if instance.amount:
151             details.append(_("Amount") + ": " + str(instance.amount))
152
153         base_object = {
154             "time": timezone.localtime(instance.start),
155             "details": details,
156             "edit_link": edit_link,
157             "model_name": instance.model_name,
158             "tags": instance.tags.all(),
159         }
160
```

```
161
162     if instance.duration > timedelta(seconds=0):
163         start_event = {
164             **base_object,
165             "event": _("%(child)s started feeding.")
166             % {"child": instance.child.first_name},
167             "time_since_prev": time_since_prev,
168             "type": "start",
169         }
170
171         end_event = {
172             **base_object,
173             "time": timezone.localtime(instance.end),
174             "event": _("%(child)s finished feeding.")
175             % {"child": instance.child.first_name},
176             "type": "end",
177             "duration": duration_string(instance.duration),
178         }
179
180         events.extend([start_event, end_event])
181     else:
182         feed_event = {
183             **base_object,
184             "event": _("%(child)s had a feeding.")
185             % {"child": instance.child.first_name},
186             "time_since_prev": time_since_prev,
187         }
188         events.append(feed_event)
```





UFOP

Code Smells

```
35 def _build_timeline_event(  
36     instance,  
37     event_time,  
38     message,  
39     details,  
40     edit_link,  
41     *,  
42     event_type=None,  
43     duration=None,  
44     time_since_prev=None,  
45     include_time_since_prev=False,  
46 ) -> TimelineEvent:  
47     """Cria um evento da timeline com os campos compartilhados."""  
48  
49     event: TimelineEvent = {  
50         "time": timezone.localtime(event_time),  
51         "event": message,  
52         "details": details,  
53         "edit_link": edit_link,  
54         "model_name": instance.model_name,  
55         "tags": instance.tags.all(),  
56     }  
57  
58     if event_type is not None:  
59         event["type"] = event_type  
60  
61     if duration is not None:  
62         event["duration"] = duration  
63  
64     if include_time_since_prev:  
65         event["time_since_prev"] = time_since_prev  
66  
67     return event
```

```
70 def _get_duration(instance):  
71     """Formata a duração somente quando ela for maior que zero."""  
72  
73     if instance.duration > timedelta(seconds=0):  
74         return duration_string(instance.duration)  
75  
76     return None  
77  
78  
79 def _get_feeding_instances(min_date, max_date, child=None):  
80     """Busca os feedings necessários para a timeline e para o cálculo anterior."""  
81  
82     yesterday = min_date - timedelta(days=1)  
83  
84     instances = Feeding.objects.filter(start__range=(yesterday, max_date)).order_by(  
85         "start"  
86     )  
87  
88     if child:  
89         instances = instances.filter(child=child)  
90  
91     return instances  
92  
93  
94 def _get_feeding_details(instance):  
95     """Monta os detalhes exibidos para um feeding."""  
96  
97     details = []  
98  
99     if instance.notes:  
100         details.append(instance.notes)  
101  
102     if instance.amount:  
103         details.append(_("Amount") + ": " + str(instance.amount))  
104  
105     return details
```





Universidade Federal de Ouro Preto
Sistemas Operacionais

Code Smells

Interpretação

- O terceiro code smell observado foi em relação ao uso de dicionários livres na representação dos eventos na timeline.
- O Python não garante que todos os dicionários tenham as mesmas chaves.
- Possibilidade de erros na nomenclatura das chaves, como usar “event_type” em uma função e apenas “type” em outra.





UFOP

Universidade Federal de Ouro Preto
Sistemas Operacionais

Code Smells

```
def _add_sleeps(min_date, max_date, events, child=None):
    instances = Sleep.objects.filter(start__range=(min_date, max_date)).order_by(
        "-start"
    )
    if child:
        instances = instances.filter(child=child)
    for instance in instances:
        details = []
        if instance.notes:
            details.append(instance.notes)
        edit_link = reverse("core:sleep-update", args=[instance.id])
        events.append(
            {
                "time": timezone.localtime(instance.start),
                "event": _("%(child)s fell asleep.")
                % {"child": instance.child.first_name},
                "details": details,
                "edit_link": edit_link,
                "model_name": instance.model_name,
                "type": "start",
                "tags": instance.tags.all(),
            }
        )

    end = {
        "time": timezone.localtime(instance.end),
        "event": _("%(child)s woke up.") % {"child": instance.child.first_name},
        "details": details,
        "edit_link": edit_link,
        "model_name": instance.model_name,
        "type": "end",
        "tags": instance.tags.all(),
    }
    if instance.duration > timedelta(seconds=0):
        end["duration"] = duration_string(instance.duration)
    events.append(end)
```





Universidade Federal de Ouro Preto
Sistemas Operacionais

UFOP

Code Smells

Interpretação

- Para solucionar o problema, foi importada a biblioteca TypedDict.
- Foi usado uma representação tipada para esses dados.
- Estrutura passou a ter um formato explícito.





Code Smells

```
21 class TimelineEvent(TypedDict):
22     """Estrutura compartilhada pelos eventos exibidos na timeline."""
23
24     time: datetime
25     details: list[str]
26     edit_link: str
27     model_name: str
28     tags: Any
29     event: NotRequired[str]
30     type: NotRequired[str | None]
31     duration: NotRequired[str]
32     time_since_prev: NotRequired[str | None]
33
```





Universidade Federal de Ouro Preto
Sistemas Operacionais

UFOP

Padrões de Projeto

Interpretação

- O primeiro padrão de projeto sugerido foi o padrão Builder.
- Anteriormente, objetos e dicionários eram montados diretamente em diferentes pontos, com várias atribuições e condições.
- Usando esse padrão, a criação de objetos é feita fora da função, que apenas chama o mesmo passando os parâmetros necessários.





UFOP

Universidade Federal de Ouro Preto
Sistemas Operacionais

Padrões de Projeto

```
8 class TimelineEventBuilder:
9     """
10     Build timeline event dictionaries with a consistent structure.
11     """
12
13     def __init__(self, instance):
14         self._event = {
15             "model_name": instance.model_name,
16             "tags": instance.tags.all(),
17         }
18         self._instance = instance
19
20     def with_time(self, value):
21         self._event["time"] = timezone.localtime(value)
22         return self
23
24     def with_message(self, message):
25         self._event["event"] = message
26         return self
27
28     def with_details(self, details):
29         self._event["details"] = details
30         return self
```





Universidade Federal de Ouro Preto
Sistemas Operacionais

Padrões de Projeto

Interpretação

- O segundo padrão sugerido foi o padrão Strategy.
- Com ele, desacoplamos o padrão de criação dos eventos do código principal.
- Dessa forma, as funções precisam apenas passar os parâmetros e a estratégia de cada função criará o evento escolhido.





UFOP

Universidade Federal de Ouro Preto
Sistemas Operacionais

Padrões de Projeto

```
11 class SleepTimelineStrategy:
12     """
13     Generate timeline events for sleep records.
14     """
15
16     def __init__(self, duration_formatter: Callable):
17         self._duration_formatter = duration_formatter
18
19     def add_events(self, min_date, max_date, events, child=None):
20         instances = Sleep.objects.filter(start__range=(min_date, max_date)).order_by(
21             "-start"
22         )
23
24         if child:
25             instances = instances.filter(child=child)
26
27         for instance in instances:
28             details = []
29             if instance.notes:
30                 details.append(instance.notes)
31
32             start_event = (
33                 TimelineEventBuilder(instance)
34                 .with_time(instance.start)
35                 .with_message(
36                     _("%(child)s fell asleep.") % {"child": instance.child.first_name}
37                 )
38                 .with_details(details)
39                 .with_edit_link("core:sleep-update")
40                 .with_type("start")
41                 .build()
42             )
43             events.append(start_event)
```





Universidade Federal de Ouro Preto
Sistemas Operacionais

Testes automatizados





Universidade Federal de Ouro Preto
Sistemas Operacionais

Testes automatizados

Interpretação

- Na parte de testes, foi desenvolvido três pequenos testes.
- Foi utilizada a ferramenta Cypress para o desenvolvimento desses testes.
- São eles:
 - Teste de tela de login;
 - Teste de inclusão de alimentação;
 - Teste de inclusão de horário de dormida.





Testes automatizados

Teste de Login

login.cy.js00:05

✓ 1 ✗ 0

Authentication

✓ logs in with valid credentials

TEST BODY

```
1 visit http://127.0.0.1:8000/login/
2 get input[name="username"]
3 -type admin
4 get input[name="password"]
5 -type admin
6 get button[type="submit"]
7 -click
8 url
  -assert expected
  http://127.0.0.1:8000/children/gustavo-dias/dashboard/ to not include /login/
  (new url) http://127.0.0.1:8000/children/gustavo-dias/dashboard/
10 get body
11 -assert expected <body> to be visible
```

http://127.0.0.1:8000/children/gustavo-dias/dashboard/1000x66088%Studio Beta

Baby BuddyDashboardTimelineChildrenMeasurementsActivitiesTimersadmin

Home / Children / Gustavo Dias / Dashboard

Timers0 timers

Recent PumpingsNone today

Last Feeding MethodBottle most recent

Recent Sleep

Last Feeding14 hours, 34 minutes ago3:37 a.m. - 4:37 a.m. Formula, Bottle

Last Sleep36 hours, 12 minutes ago6 a.m. 8 hours, 0 minutes

Statistics0 minutes Feeding frequency (past 3 days)

Last Diaper ChangeNone

Last PumpingNone

Last MedicationNone

Recent Feedings100 2 feedings today





Testes automatizados

Teste de Horário de Comida

feeding.cy.js00:02

✓ 1 ✗ -- ○ --

✓ registers a feeding

```
2 get input[name="username"]
3 -type admin
4 get input[name="password"]
5 -type admin
6 get button[type="submit"]
7 -click
8 url
  -assert expected
  http://127.0.0.1:8000/children/gustavo-dias/dashboard/ to not include /login/
  (new url) http://127.0.0.1:8000/children/gustavo-dias/dashboard/
```

TEST BODY

```
1 visit http://127.0.0.1:8000/feedings/add/
2 get body
3 -assert expected <body> to be visible
  (new url) http://127.0.0.1:8000/feedings/
```

http://127.0.0.1:8000/feedings/1000x660 88%Studio Beta

Baby BuddyDashboardTimelineChildrenMeasurementsActivitiesTimersadmin

Home / Feedings

Feeding entry for Gustavo Dias added!

FeedingsAdd FeedingAdd Bottle Feeding

Filters

Actions	Date	Method	Type	Amt.	Duration	Previous	Tags
 	Today, 6:16 p.m.	Bottle	Formula			14 hours, 38 minutes ago	-
 	Today, 3:37 a.m.	Bottle	Formula		1 hour, 0 minutes	0 minutes ago	-
 	Today, 3:37 a.m.	Bottle	Breast milk	100.0			-





Testes automatizados

Teste de Horário de Dormida

sleep.cy.js00:03

✓ 1 ✗ -- ○ --

✓ registers a completed sleep period

8 url
- assert expected
9 http://127.0.0.1:8000/children/gustavo-dias/dashboard/ to not include /login/
(new url) http://127.0.0.1:8000/children/gustavo-dias/dashboard/
TEST BODY
1 visit http://127.0.0.1:8000/sleep/add/
2 get input[name="start"]
3 -clear
4 -type 2026-07-15T22:00
5 get input[name="end"]
6 -clear
7 -type 2026-07-16T06:00
8 -contains button, Submit
9 -click
(new url) http://127.0.0.1:8000/sleep/

http://127.0.0.1:8000/sleep/1000x66088%Studio Beta

Baby BuddyDashboardTimelineChildrenMeasurementsActivitiesTimersadmin

Home / Sleep

Sleep entry for Gustavo Dias added!

SleepAdd Sleep

Filters

Actions	Start	End	Duration	Previous	Nap	Tags
 	Jul 15, 10 p.m.	Jul 16, 6 a.m.	8 hours, 0 minutes	1 day, 23 hours ago		-
 	Jul 13, 10:08 p.m.	Jul 14, 2:08 a.m.	4 hours, 0 minutes	3 days, 7 hours ago		-
 	Jul 10, 2:31 p.m.	Jul 10, 4:31 p.m.	2 hours, 0 minutes	12 hours, 4 minutes ago		-
 	Jul 10, 2:27 a.m.	Jul 10, 6:27 a.m.	4 hours, 0 minutes			-





UFOP

Universidade Federal de Ouro Preto
Sistemas Operacionais

Pipeline CI/CD





Universidade Federal de Ouro Preto
Sistemas Operacionais

UFOP

Pipeline

Interpretação

- Como melhoria na pipeline CI/CD, foi proposto a implementação da ferramenta pip-audit.
- Sempre que ocorre um Push ou Pull Request, o GitHub cria um ambiente Ubuntu.
- Após, configura o Python 3.10, instala o Pipenv e todas as dependências do projeto e, por fim, executa o pip-audit.
- Caso alguma biblioteca possua vulnerabilidade conhecida, a pipeline falha e alerta o desenvolvedor.

